XMimic: Learning Whole-Body Motion Tracking in Under a Minute

Nico Bohlinger[1], Jan Peters[1,2]

***Abstract*— We present XMimic, a JAX implementation of SuperMimic-style reinforcement learning that learns to track all 40 LAFAN1 motion clips with a single Unitree G1 policy in 0.91 minutes on four H100 GPUs. XMimic compiles whole training iterations into single GPU programs and splits each iteration across up to four GPUs that work at the same time. With the same algorithm, hyperparameters, and evaluation as BlinkMimic, XMimic reaches the same target 1.3 times faster on one GPU and 3.5 times faster on four GPUs. The code is available at https://github.com/nico-bohlinger/xmimic.**

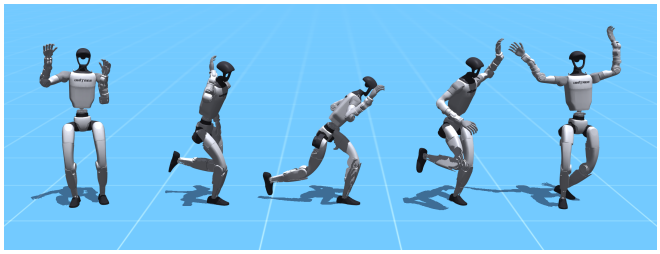


Fig. 1. We train a single policy that tracks all 40 LAFAN1 motion clips with the Unitree G1 in 0.91 minutes on four H100 GPUs. The render shows the policy at five moments of a dance clip after 54 seconds of training.

I. INTRODUCTION

SuperMimic [1] learns whole-body motion tracking for humanoid robots by backpropagating through rollouts that are imagined in a learned and differentiable world model, similar to SuperTrack [2] for simulated characters. The idea was quickly taken up by the community. Starting from the unofficial reimplementation supermimic_vibed [3], InstantTrack [4] and BlinkMimic [5] reduced the time to track all 40 LAFAN1 clips [6] with the Unitree G1 from about 1.5 hours to about 3 minutes on a single consumer GPU. However, these implementations still run the training loop in PyTorch as a long chain of small GPU kernels that are launched and synchronized by Python. Their speed is therefore bound by the host rather than the GPU, and BlinkMimic runs no faster on an NVIDIA H100 than on an RTX 4090. Additionally, a single GPU has to run the simulation, the world-model fit, and the policy update one after another.

This is why we propose XMimic, which implements the BlinkMimic algorithm in JAX [7] with the MuJoCo Warp simulator [8], following the idea of InstantMimic [9] to keep the whole training loop on the GPU. XMimic compiles complete training iterations into single Accelerated Linear Algebra (XLA) programs and splits each iteration across up to four GPUs, which collect data, fit the world model, and update the policy at the same time. With BlinkMimic's hyperparameters and evaluation, XMimic reaches BlinkMimic's target in 2.48 minutes on a single H100 GPU and in 0.91 minutes on four, which is 1.3 and 3.5 times faster than BlinkMimic on the same hardware.

[1]Department of Computer Science, Technical University of Darmstadt, Germany. [2]Robotics Institute Germany (RIG); German Research Center for AI (DFKI); hessian.AI.
Corresponding author: nico.bohlinger@tu-darmstadt.de

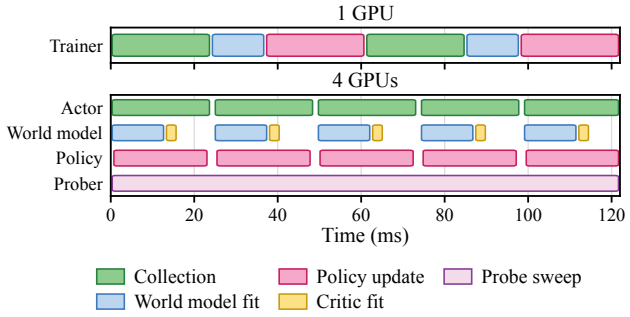


Fig. 2. Training iterations on one GPU (top) and split across four GPUs (bottom), approximately to scale with times measured on H100 GPUs. On four GPUs, the actor, the world-model learner, and the policy learner work at the same time on the results of the previous iteration, while the fourth GPU runs the probe sweeps.

II. MOTION TRACKING IN UNDER A MINUTE

We build upon BlinkMimic [5] and keep its algorithm and hyperparameters, so that all speedups come from the implementation. In every iteration, BlinkMimic collects about 2.5 control steps in each of 4096 parallel environments, fits the world model five times on 8-step windows from the replay buffer, and updates the policy by backpropagating through 32 imagined steps (see Figure 2). The policy and the world model are trained with Muon [10]. The deployed policy is an exponential moving average of the policy weights, and a critic bootstraps the returns of the imagined rollouts. Every 125 iterations, a probe sweep plays the deployed policy on all clips to find the places where it fails, and part of the episodes then start right before them.

A. Compiled Training Iterations

To keep the GPU busy, XMimic runs every part of the iteration in JAX, including the simulator, which is MuJoCo Warp through MuJoCo XLA (MJX) [8], [11]. On a single GPU, 50 iterations are compiled into one XLA program, so nothing returns to Python in between, and checkpoints are written from a background thread. We also make the learner itself faster. As in PyTorch's mixed precision, the outputs of

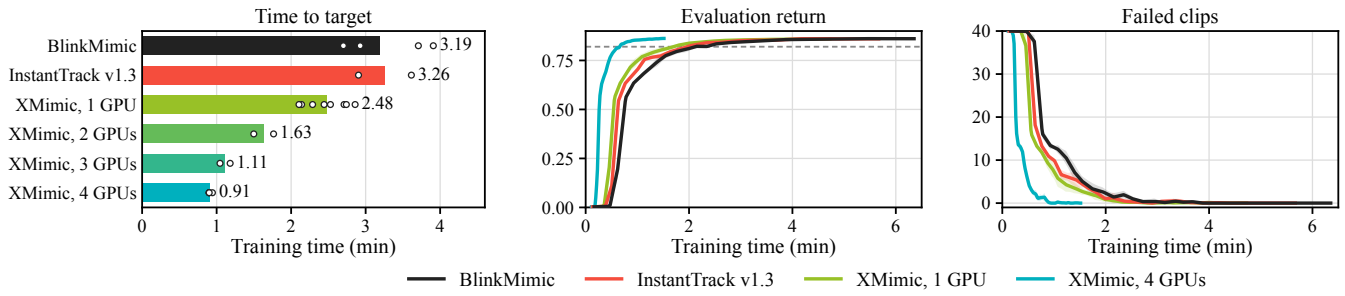


Fig. 3. Time to target (left), evaluation return (middle), and number of failed clips (right) on H100 GPUs, measured in training time without compilation. The bars show the mean and the dots the individual runs. The curves show the mean and standard deviation over 5 runs of BlinkMimic, 2 of InstantTrack v1.3, and 8 and 4 of XMimic on one and four GPUs. The dashed line marks the return target of 0.82.

matrix multiplications are stored in bfloat16, so the backward pass runs on bfloat16 tensor cores as well. We further hoist all computation that does not depend on the imagined state out of the 32-step loop, compute the weight gradients of each layer with a single matrix multiplication, and batch the Muon updates of weights with the same shape. This reduces the time of a policy update on an H100 from 41.6 to 25.9 ms, while its gradients stay within the rounding error of mixed precision.

B. Splitting the Iteration across GPUs

With more GPUs, XMimic splits the iteration into three parts that run at the same time (see Figure 2). An actor collects data in the simulator, a world-model learner fits the world model and the critic, and a policy learner runs the backpropagation through imagined rollouts. Each part works with the results of the others from the previous iteration, so the iteration time drops from the sum to the maximum of the three parts,

$$T_{\text{single}} = T_{\text{c}} + T_{\text{w}} + T_{\text{p}}, \qquad T_{\text{split}} \approx \max(T_{\text{c}}, T_{\text{w}}, T_{\text{p}}), \quad (1)$$

where T_{c}, T_{w}, and T_{p} are the times of the collection, the world-model fit, and the policy update, respectively. The price is that the actor collects with a policy that is one iteration old, and the policy learner imagines with a world model that is two iterations old. The fourth GPU runs the probe sweeps, which halve the training speed of a single GPU while they run. Finally, we keep the host out of the way. The actor runs in its own Python thread, the host computes the number of collection steps of every iteration in advance so that no launch waits for the GPU, and Python's garbage collector is frozen after setup. Together, these changes raise the speed on four GPUs from 20 to about 40 iterations per second.

III. EXPERIMENTS

We follow the task and evaluation of BlinkMimic, where a single policy learns to track all 40 LAFAN1 clips, 2.45 hours of motion retargeted to the 29 joints of the Unitree G1. We compare XMimic with BlinkMimic and InstantTrack v1.3, both unmodified on their PyTorch and mjlab [12] stack, and run every method for 3500 iterations on H100 GPUs of the same cluster. Every 100 iterations, we evaluate the deployed policy on all clips with four replicas. As in BlinkMimic, the target is reached once the normalized return is at least 0.82 with no failed clip for three consecutive checkpoints. We report the training time to this target without compilation for all methods. The only difference in the simulation is that XMimic trains with at most 256 constraint rows per environment instead of 500, which is 7% faster per physics step and only rarely exceeded.

Figure 3 shows the time to target and the learning curves of all methods. On a single GPU, XMimic reaches the target in 2.48 minutes, 1.3 times faster than BlinkMimic with 3.19 minutes, as it runs 12.2 instead of 9.1 iterations per second. With more GPUs, the time drops to 1.63, 1.11, and 0.91 minutes on two, three, and four GPUs. Despite the older policy and world model, XMimic on four GPUs needs only about 10% more iterations than BlinkMimic and runs 38 instead of 9.1 iterations per second, which makes it 3.5 times faster. It trades a little more total compute for this speed, as the four GPUs together use 3.6 GPU-minutes to reach the target against 3.2 for BlinkMimic. All methods end at the same evaluation return of 0.86, so XMimic learns the same tracking quality, only sooner.

IV. CONCLUSION

We presented XMimic, an implementation of SuperMimic-style motion tracking in JAX that compiles whole training iterations and splits them across up to four GPUs. XMimic learns to track all 40 LAFAN1 clips with the Unitree G1 in 0.91 minutes on four H100 GPUs, 3.5 times faster than BlinkMimic with the same algorithm and hyperparameters. With four GPUs, the collection and the policy update each take almost the whole iteration of 24.7 ms and now bound the training speed. In future work, we will split the collection across more GPUs and use the short training times to scale XMimic to larger motion datasets.

ACKNOWLEDGMENT

We acknowledge EuroHPC Joint Undertaking for awarding us access to MeluXina at LuxProvide, Luxembourg, Vega at IZUM, Slovenia, and Arrhenius at NAISS, Sweden, through the Development Access project EHPC-DEV-2026D08-135. The authors gratefully acknowledge the HPC RIVR consortium and EuroHPC JU for funding this research

by providing computing resources of the HPC system Vega at the Institute of Information Science. Some of the experiments were performed on the Luxembourg national supercomputer MeluXina, and the authors gratefully acknowledge the LuxProvide teams for their expert support. Computational resources were provided by the National Academic Infrastructure for Supercomputing in Sweden (NAISS), funded by the Swedish Research Council. The authors gratefully acknowledge the computing time granted by the KISSKI project. The calculations for this research were conducted with computing resources under the project kisski-merl. The authors gratefully acknowledge the computing time provided to them on the high-performance computer Lichtenberg II at the NHR Center NHR4CES at TU Darmstadt (project number p0028682). This is funded by the Federal Ministry of Research, Technology and Space, and the state governments participating on the basis of the resolutions of the GWK for national high performance computing at universities. The authors gratefully acknowledge the computing time made available to them on the high-performance computers Otus [13] and Noctua 2 [14] (project hpc-prf-smerl) at the NHR Center Paderborn Center for Parallel Computing (PC2). This center is jointly supported by the Federal Ministry of Research, Technology and Space and the state governments participating in the National High-Performance Computing (NHR) joint funding program (www.nhr-verein.de/en/our-partners). Some of the experiments ran on the hessian.AI cluster 43, and we gratefully acknowledge support from the hessian.AI Service Center (funded by the Federal Ministry of Research, Technology and Space, BMFTR, grant no. 16IS22091) and the hessian.AI Innovation Lab (funded by the Hessian Ministry for Digital Strategy and Innovation, grant no. S-DIW04/0013/003).

REFERENCES

[1] Q. Liao, Y. Shao, J. Long, K. Zakka, R. Horowitz, P. Abbeel, and K. Sreenath, "SuperMimic: Scaling humanoid whole-body control with one differentiable world model," *arXiv preprint arXiv:2610.28480*, 2026.

[2] L. Fussell, K. Bergamin, and D. Holden, "SuperTrack: Motion tracking for physically simulated characters using supervised learning," *ACM Transactions on Graphics (Proceedings of ACM SIGGRAPH Asia)*, vol. 40, no. 6, 2021.

[3] C. Zhang, "supermimic_vibed: An unofficial reimplementation of SuperMimic," 2026. [Online]. Available: https://github.com/zita-ch/supermimic_vibed

[4] Huy-Quang-Dao, "InstantTrack: Whole-body motion tracking for the Unitree G1 in 15 minutes on one GPU," 2026. [Online]. Available: https://github.com/Huy-Quang-Dao/instantTrack

[5] F. Chen, "BlinkMimic: Whole-body motion tracking for the Unitree G1 in minutes on one GPU," 2026. [Online]. Available: https://github.com/fuyichen0725/blinkmimic

[6] F. G. Harvey, M. Yurick, D. Nowrouzezahrai, and C. Pal, "Robust motion in-betweening," *ACM Transactions on Graphics (Proceedings of ACM SIGGRAPH)*, vol. 39, no. 4, pp. 60:1–60:12, 2020.

[7] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, "JAX: composable transformations of Python+NumPy programs," 2018. [Online]. Available: http://github.com/jax-ml/jax

[8] Google DeepMind and NVIDIA, "MuJoCo Warp: GPU-accelerated MuJoCo with NVIDIA Warp," 2025. [Online]. Available: https://github.com/google-deepmind/mujoco_warp

[9] I. Choi, G. Leem, and J. Won, "InstantMimic: A high performance system for learning physics-based skills in seconds," *arXiv preprint arXiv:2609.09821*, 2026.

[10] K. Jordan, Y. Jin, V. Boza, J. You, F. Cesista, L. Newhouse, and J. Bernstein, "Muon: An optimizer for hidden layers in neural networks," 2024. [Online]. Available: https://kellerjordan.github.io/posts/muon/

[11] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012, pp. 5026–5033.

[12] K. Zakka, Q. Liao, B. Yi, L. Le Lay, K. Sreenath, and P. Abbeel, "mjlab: A lightweight framework for GPU-accelerated robot learning," *arXiv preprint arXiv:2601.22074*, 2026.

[13] S. Ehtesabi, M. Hossain, T. Kenter, A. Krawinkel, H. Nitsche, L. Ostermann, C. Plessl, H. Riebler, S. Rohde, R. Schade, M. Schwarz, J. Simon, N. Winnwa, A. Wiens, and X. Wu, "Otus Supercomputer," Paderborn Center for Parallel Computing (PC2), Paderborn, Germany, PC2 Technical Report PC2TR-2025-1, Dec. 2025.

[14] C. Bauer, T. Kenter, M. Lass, L. Mazur, M. Meyer, H. Nitsche, H. Riebler, R. Schade, M. Schwarz, N. Winnwa, A. Wiens, X. Wu, C. Plessl, and J. Simon, "Noctua 2 supercomputer," *Journal of large-scale research facilities JLSRF*, vol. 9, 2024.